

Redpwn 2020 Crypto - Pseudo-key

deuterium — 2020-06-27

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/redpwn/crypto/pseudo-key/2020-06-27-redpwn-2020-pseudo-key.html>

Abstract

CTF writeup: A Vigenere key encrypts itself in redpwnCTF 2020's Pseudo-key, leaving 2,048 candidates. Self-care would have involved a different key.

Pseudo key

Description

Keys are not always as they seem...
Note: Make sure to wrap the plaintext with flag{} before you submit!

Files

- [pseudo-key-output.txt](#)
- [pseudo-key.py](#)

Lets take a quick look at the contents of [pseudo-key.py](#)

```
#!/usr/bin/env python3

from string import ascii_lowercase

chr_to_num = {c: i for i, c in enumerate(ascii_lowercase)}
num_to_chr = {i: c for i, c in enumerate(ascii_lowercase)}

def encrypt(ptxt, key):
    ptxt = ptxt.lower()
    key = ''.join(key[i % len(key)] for i in range(len(ptxt))).lower()
    ctxt = ''
    for i in range(len(ptxt)):
        if ptxt[i] == '_':
            ctxt += '_'
            continue
        x = chr_to_num[ptxt[i]]
        y = chr_to_num[key[i]]
        ctxt += num_to_chr[(x + y) % 26]
    return ctxt

with open('flag.txt') as f, open('key.txt') as k:
    flag = f.read()
    key = k.read()

ptxt = flag[5:-1]

ctxt = encrypt(ptxt, key)
```

```
pseudo_key = encrypt(key,key)

print('Ciphertext:',ctxt)
print('Pseudo-key:',pseudo_key)
```

Encryption is just vignere cipher over ASCII-lower with a given key.

NOTE: underscores are skipped and added as such

But whats the challenge here?

The key is encrypted with the key itself and we are provided the encrypted key (called Pseudo-key) `iigesssaemk`

Treating the alphabets as numbers from 0-26

$$ciphertext_i = plaintext_i + key_i \mod 26$$

Where key_i essentially suggests that key is duplicated again and again to match the $plaintext$ length

And hence, if we encrypt the key with key itself, we get,

$$pseudoKey_i = key_i + key_i \mod 26$$

$$pseudoKey_i = 2key_i \mod 26$$

$$key_i = pseudoKey_i/2 \text{ or } pseudoKey_i/2 + 13$$

Since the length of $pseudoKey$ is 11, we have $2^{11} = 2048$ possibilities, which we could simply go through and check if we get a valid flag

```
from itertools import product
ct = "z_jjaoo_rljlhr_gauf_twv_shaqzb_ljtyut"
pseudo_key = "iigesssaemk"

pseudo_key_to_num = [ord(i)-97 for i in pseudo_key]
possible_keys = [ (i//2, i//2+13) for i in pseudo_key_to_num ]

ct_to_num = [ord(i)-97 if ord(i) >= 97 else i for i in ct ]

def decrypt(ct,key):
    keylen = len(key)
    return "".join( ct[i] if str(ct[i])=='_' else chr((ct[i] - key[i%keylen] +26)%26
+97 ) for i in range(len(ct)) )

for key in product(*possible_keys):
    decrypted = decrypt(ct_to_num, key)
    print(decrypted, "".join(chr(i+97) for i in key ))
```

Scrolling over the produced ciphertext, one could easily identify the correct flag

```
...
i_guess_pfrudo_keyf_nee_pseudb_frcure redpwwwacgs
i_guess_pseudo_keyf_are_pseudb_secure redpwwwactf
i_guess_psrudo_keyf_aee_pseudb_srcure redpwwwacts
i_guess_cfeudo_keyf_nre_pseudb_fecure redpwwwapgf
i_guess_cfrudo_keyf_nee_pseudb_frcure redpwwwapgs
i_guess_cseudo_keyf_are_pseudb_secure redpwwwaptf
i_guess_csrudo_keyf_aee_pseudb_srcure redpwwwapts
i_guess_pfeudo_keys_nre_pseudo_fecure redpwwwncgf
i_guess_pfrudo_keys_nee_pseudo_frcure redpwwwncgs
i_guess_pseudo_keys_are_pseudo_secure redpwwwnctf
i_guess_psrudo_keys_aee_pseudo_srcure redpwwwncts
i_guess_cfeudo_keys_nre_pseudo_fecure redpwwwnpgf
i_guess_cfrudo_keys_nee_pseudo_frcure redpwwwnpgs
i_guess_cseudo_keys_are_pseudo_secure redpwwwnptf
i_guess_csrudo_keys_aee_pseudo_srcure redpwwwnpts
...
```

The key appears to be redpwwwnctf

flag-i'guess'pseudo'keys'are'pseudo'secure"

| Interactive controls are omitted from this PDF. See the canonical HTML version.